

A Benchmark Task Explanations

A.1 Design Principles

The benchmark uses three families of tasks:

- **Short-term tasks:** immediate tactical choices with direct board consequences.
- **Medium-term tasks:** action prioritization and resource conversion decisions.
- **Long-term tasks:** strategic race commitments whose value unfolds over several turns.

The main design goal is to compare decisions with the best legal alternative available at that moment (based on a heuristic approach), while keeping a threshold value to determine success or failure on the individual task. This system rewards near-optimal play when several choices are reasonable, and avoids placing fail labels on decisions that were reasonable but not perfectly aligned with the benchmark heuristic.

A.2 Shared Heuristic Building Blocks

A.2.1 Production expectancy

Some dice numbers have a higher probability of being rolled than others. Specifically, the probability weights are:

$$2, 12 \mapsto 1, \quad 3, 11 \mapsto 2, \quad 4, 10 \mapsto 3, \quad 5, 9 \mapsto 4, \quad 6, 8 \mapsto 5.$$

7 is a number used for the robber and does not produce resources. To compute the expected value of the resources gained on the next turn from a current settlement vertex,

$$\begin{aligned} \text{productionWeight} &= \sum \text{adjacent_hex_probability_weights}, \\ \text{resourceProductionExpectancy} &= \min\left(1, \frac{\text{productionWeight}}{15}\right). \end{aligned}$$

The denominator 15 is the theoretical maximum because a vertex touches at most three hexes and the maximum per-hex weight is 5.

A.2.2 Resource diversity

Each settlement vertex is adjacent to up to three hexes, which can produce up to three different resource types (`distinct_adjacent_resource_types`).

Resource diversity is defined as

$$\text{resourceDiversity} = \min\left(1, \frac{\text{distinct_adjacent_resource_types}}{3}\right).$$

It matters because diverse starts reduce dependency on trading, improve build robustness, and lower the risk of bottlenecking on a single missing resource.

A.2.3 Expansion access

In crowded boards, some settlement locations have more future legal road placements than others, which translates into better expansion potential and better survivability in case of unfavorable dice rolls or robber placements. For a settlement candidate, the evaluator computes incident edges, keeps only edges where a road could legally be placed, and defines

$$\text{expansionAccess} = \min \left(1, \frac{\text{legal_outgoing_edges}}{3} \right).$$

A.2.4 Port synergy

We use a discrete port heuristic:

- specific 2:1 port $\rightarrow 1.0$
- generic 3:1 port $\rightarrow 0.7$
- no port $\rightarrow 0.0$

Ports matter because they convert surplus resources into missing ones; specific ports are usually stronger due to higher exchange efficiency.

A.2.5 Build plan base scores

Several tasks construct plan candidates for road, settlement, city, and devCard. Each plan has a `baseScore`: the current heuristic value of that build family before applying affordability or distance-to-completion discounts. The shared definitions are:

- road: the maximum score among currently legal road placements under the road-placement rubric from Section A.3.3.
- settlement: the maximum score among currently legal settlement placements under the settlement rubric from Section A.3.2.
- city: the maximum score among legal city upgrades using $0.65 \cdot \text{production} + 0.10 \cdot \text{diversity} + 0.05 \cdot \text{portSynergy} + 0.20$.
- devCard: 0 if the development-card deck is unavailable; otherwise $\max(0.2, 0.55 \cdot (0.5 + 0.5 \cdot \text{cityPlanScore}))$, where `cityPlanScore` is the current best city-upgrade score computed with the city formula in this list.

A.2.6 Resource need and surplus

Resource need and surplus are used in trade and bank-trade scoring.

For each candidate plan and resource r , `cost[r]` is the number of cards of resource r required by the plan, `hand[r]` is the player's current count of resource r , and

$$\text{missing}[r] = \max(0, \text{cost}[r] - \text{hand}[r]).$$

`totalMissing` is the total number of resource cards still needed for the plan:

$$\text{totalMissing} = \sum_r \text{missing}[r].$$

Each plan is weighted according to how many resources are needed to achieve the plan:

$$\text{planWeight} = \frac{\text{baseScore}}{1 + 0.75 \cdot \text{totalMissing}},$$

where `baseScore` uses the shared build-plan definition from Section A.2.5. For each resource r ,

$$\text{need}[r] += \text{planWeight} \cdot \frac{\text{missing}[r]}{\max(1, \text{totalMissing})}.$$

For surplus, the evaluator reserves part of the current hand for weighted plans:

$$\text{reserved}[r] = \min(\text{hand}[r], \sum \text{planWeight} \cdot \text{cost}[r]),$$

then treats any unreserved cards of resource r as surplus. The surplus map is normalized by the largest unreserved resource count in the hand:

$$\text{surplus}[r] = \frac{\max(0, \text{hand}[r] - \text{reserved}[r])}{\max(1, \max_s \max(0, \text{hand}[s] - \text{reserved}[s]))},$$

where s ranges over all resource types. This avoids earlier failure modes where a resource such as ore looked important merely because it was scarce in hand, even when the board strongly favored early road and settlement expansion.

A.3 Task-by-Task Explanations

A.3.1 initial-settlement-location-selection

`Score = 0.40 · production + 0.25 · diversity + 0.20 · expansionAccess + 0.15 · portSynergy.`

This rubric uses the shared `production`, `diversity`, `expansionAccess`, and `portSynergy` definitions from Sections A.2.1–A.2.4. Production receives the highest weight because early income determines the opening trajectory, while the other terms preserve resource mix, expansion geometry, and port upside. The evaluator applies the rubric above to the shared settlement-quality quantities and clamps the final score to $[0, 1]$.

A.3.2 settlement-location-selection

This task uses the same scoring system as the initial-settlement task in Section A.3.1. The core difference between initial and subsequent location selection is that the initial settlement location selection task has a much higher weightage in the overall benchmark. This reflects the fact that the first settlement sets the foundation for the entire game and is a critical decision point, while subsequent settlement placements, although still important, are often more constrained by the existing board state and may have less impact on the overall game trajectory.

A.3.3 road-placement-direction

$$\text{Score} = 0.50 \cdot \text{futureReachability} + 0.35 \cdot \text{reachableValue} + 0.15 \cdot \text{blockingValue}.$$

Where:

- `futureReachability` is the forward vertex's onward legal-road count, clamped after division by 3.
- `reachableValue` is the forward vertex's weighted settlement value:

$$\text{reachableValue} = 0.5 \cdot \text{production} + 0.3 \cdot \text{diversity} + 0.2 \cdot \text{portSynergy}.$$

- `blockingValue` is the maximum blocking score over the candidate road's two endpoints and all opponents present at those endpoints.

For each endpoint-opponent pair, the evaluator ignores endpoints occupied by the acting player and endpoints occupied by a different opponent. It then computes:

$$O_{\text{branch}} = \text{open incident edges excluding the candidate road},$$

$$\text{branchScarcity} = \text{clamp} \left(\frac{2 - O_{\text{branch}}}{2} \right),$$

$$\text{vertexOpportunity} = \begin{cases} 0.8, & \text{if the opponent already owns the endpoint,} \\ 1.0, & \text{if the endpoint is an open legal settlement site,} \\ 0.35, & \text{otherwise} \end{cases}$$

Let R_{adj} be the number of opponent roads adjacent to the endpoint, B_{opp} be 1 if the opponent owns the endpoint and 0 otherwise, L_{opp} be the opponent's road length, and H_{opp} be 1 if the opponent has Longest Road and 0 otherwise.

$$\text{presenceStrength} = \text{clamp} \left(\frac{R_{\text{adj}} + B_{\text{opp}}}{2} \right),$$

$$\text{roadPressure} = \text{clamp} \left(\frac{L_{\text{opp}} + 2 \cdot H_{\text{opp}}}{8} \right).$$

The candidate blocking score is

$$\begin{aligned} \text{candidateBlocking} = & \text{clamp}(0.50 \cdot \text{branchScarcity} + 0.30 \cdot \text{vertexOpportunity} \\ & + 0.10 \cdot \text{presenceStrength} + 0.10 \cdot \text{roadPressure}). \end{aligned}$$

and `blockingValue` is the largest `candidateBlocking`, or 0 if no opponent is present at either endpoint. The current implementation therefore recognizes common denial plays without running a full tactical search. Future reachability is weighted highest because roads are primarily investment moves. Reachable intersection value matters because roads derive value from the future settlement points they unlock. Blocking value is tactical and useful, but not every road is a denial move.

A.3.4 knight-card-playing-decision

Define:

- `robberThreatLevel` measures how costly the current robber placement is for the player.
- `resourceNeed` is the largest entry in the goal-aware resource-need map from Section A.2.6.
- `armyProgressRatio` measures progress toward Largest Army after playing the Knight:

$$\text{robberThreatLevel} = \begin{cases} \text{clamp}(\text{robberNumberWeight}/5), & \text{if the robber blocks the player} \\ 0.15, & \text{otherwise} \end{cases}$$

$$\text{resourceNeed} = \max_r \text{need}[r].$$

This captures how valuable a Knight steal is when the player is missing an important resource for a near-term plan. Let K_{self} be the player's current Knight count and K_{opp} be the largest opponent Knight count. Then:

$$\text{armyProgressRatio} = \text{clamp} \left(\frac{K_{\text{self}} + 1}{\max(3, K_{\text{opp}} + 1)} \right).$$

Playing a Knight is most valuable when the robber is harming the player, when stealing a needed resource is likely to matter, and when Largest Army progress matters. The score does not include resource hand size because the Knight card has already been purchased and playing it does not reduce seven-risk for the player's resource hand.

The live scores are

$$\begin{aligned} \text{playNowScore} &= 0.45 \cdot \text{robberThreatLevel} + 0.35 \cdot \text{resourceNeed} \\ &\quad + 0.20 \cdot \text{armyProgressRatio} \\ \text{holdScore} &= \text{clamp}(0.25 + 0.75 \cdot (1 - \text{playNowScore})). \end{aligned}$$

A.3.5 build-vs-save-decision

This decision compares the expected value of spending now against preserving option value for later, incorporating immediate gains, future unlock potential, and seven-risk. The runtime constructs plan candidates for road, settlement, city, and devCard. Each plan uses the shared baseScore definitions from Section A.2.5, a cost vector, and a missingCount. missingCount is the total number of missing resource cards for the plan's cost vector. The discounted future value is

$$\text{effectiveScore} = \frac{\text{baseScore}}{1 + 0.75 \cdot \text{missingCount}}.$$

Each affordable build option gets $\text{score} = \text{clamp}(\text{baseScore})$. The save alternative is

$$B_{\text{eff}} = \text{best unaffordable effectiveScore}.$$

$$\text{saveScore} = \text{clamp} \left(\max(0.35, B_{\text{eff}} + S_7) \right),$$

where S_7 is 0.1 when $\text{resourceTotal} > 7$ and 0 otherwise.

A.3.6 year-of-plenty-card-playing-decision

Year of Plenty is strongest when it immediately unlocks a valuable build, repairs bottlenecks, and improves future resource flexibility. The runtime waits until both chosen resources are known, then scores every unordered resource pair using the shared plan table from Section A.2.5. It computes beforeBest as the largest baseScore affordable before the card, or 0 if no plan is affordable. It then simulates each candidate resource pair and computes afterBest the same way from the simulated post-card hand. For each pair:

$$\text{buildUnlock} = \text{clamp}(\max(0, \text{afterBest} - \text{beforeBest}) + U_{\text{imm}}),$$

where U_{imm} is `afterBest` if `afterBest > beforeBest` and 0 otherwise.

$$\text{resourceScarcity} = \text{clamp} \left(\frac{\text{need}[r_1] + \text{need}[r_2]}{2} \right),$$

where r_1 and r_2 are the two resources in the candidate pair.

$$\text{longtermDiversityGain} = \text{clamp} \left(\frac{\text{distinctAfter} - \text{distinctBefore}}{2} \right).$$

The final score is

$$\begin{aligned} \text{Score} = & \text{clamp}(0.50 \cdot \text{buildUnlock} + 0.35 \cdot \text{resourceScarcity} \\ & + 0.15 \cdot \text{longtermDiversityGain}). \end{aligned}$$

A.3.7 monopoly-card-playing-decision

Monopoly is strongest when opponents likely hold a concentrated resource, the acting player currently needs that resource, and the resulting swing unlocks a build. For each candidate called resource, the server sums opponents' actual holdings of that resource, computes `beforeBest`, simulates adding all stolen copies, and recomputes `afterBest`. It then uses:

$$\text{estimatedOpponentHoldings} = \text{clamp} \left(\frac{C_r}{C_{\text{max}}} \right),$$

where C_r is the total opponent copies of the called resource and C_{max} is the largest opponent holding count across any resource.

$$\text{personalResourceNeed} = \text{clamp}(\text{need}[r]).$$

$$\text{buildUnlock} = \text{clamp}(\max(0, \text{afterBest} - \text{beforeBest}) + U_{\text{imm}}),$$

where U_{imm} is `afterBest` if `afterBest > beforeBest` and 0 otherwise. The resource score is

$$\begin{aligned} \text{resourceScore} = & \text{clamp}(0.55 \cdot \text{estimatedOpponentHoldings} + 0.30 \cdot \text{personalResourceNeed} \\ & + 0.15 \cdot \text{buildUnlock}). \end{aligned}$$

Monopoly plays emit a benchmark payload over the five possible resource calls. The implementation uses actual hidden-opponent holdings available on the server.

A.3.8 road-building-card-playing-decision

The central question is whether playing Road Building now is more valuable than holding it for a better timing window. The evaluator scores every currently legal road placement using the road-placement heuristic from Section A.3.3, sorts scores descending, averages the best two legal road scores (or one if only one exists), and computes

$$\text{playNowScore} = \text{clamp}(0.70 \cdot \text{topTwoAverage} + 0.30 \cdot \text{longestRoadLeverage}),$$

where

$$\text{longestRoadLeverage} = \text{clamp} \left(\frac{(L_{\text{self}} + 2) - L_{\text{opp}} + 3}{6} \right).$$

Here L_{self} is the player's current road length and L_{opp} is the best opponent road length. The hold alternative is

$$\text{holdScore} = \text{clamp}(0.30 + 0.70 \cdot (1 - \text{playNowScore})).$$

A.3.9 robber-victim-selection

This task is evaluated as an option ratio over legal victims. Victim selection is a discrete choice whose value depends on leader pressure, theft value, and visible board strength. The visible leader is the player with the highest public score:

$$\text{visibleLeaderScore} = \text{visibleVP} + 2 \cdot \text{hasLongestRoad} + 2 \cdot \text{hasLargestArmy}.$$

If multiple players tie, the implementation keeps the first tied player in turn-order/index order. For each legal victim v , the score is

$$\text{victimScore}(v) = \text{clamp}(\text{leaderBase}(v) + \text{handValue}(v) + \text{visibleStrength}(v)),$$

where

$$\begin{aligned} \text{leaderBase}(v) &= \begin{cases} 0.55, & \text{if } v \text{ is the visible leader,} \\ 0.25, & \text{otherwise,} \end{cases} \\ \text{handValue}(v) &= \min\left(0.35, \frac{\text{resourceTotal}(v)}{20}\right), \\ \text{visibleStrength}(v) &= \min\left(0.10, \frac{\text{visibleScore}(v)}{20}\right). \end{aligned}$$

A.3.10 robber-placement

$$\begin{aligned} \text{Score} &= 0.35 \cdot \text{hurtLeader} + 0.30 \cdot \text{productionDamage} \\ &\quad + 0.20 \cdot \text{theftValue} + 0.15 \cdot \text{selfHarmAvoidance}. \end{aligned}$$

For a candidate hex h , the evaluator uses the same visible leader definition from Section A.3.9. It identifies all opponents on that hex and computes each opponent's strength proxy:

$$\text{strength}(p) = \min\left(1, \frac{\text{visibleVP}(p) + \text{resourceTotal}(p)/8}{5}\right).$$

Let $w(h)$ be the dice-number probability weight of the hex. The damage terms are:

$$\begin{aligned} \text{hurtLeader} &= \min\left(1, \frac{w(h) \sum_{p \in h, p=\text{visible leader}} \text{strength}(p)}{3}\right), \\ \text{productionDamage} &= \min\left(1, \frac{w(h) \sum_{p \in h, p \neq \text{self}} \text{strength}(p)}{5}\right), \\ \text{theftValue} &= \max_{p \in h, p \neq \text{self}} \min\left(1, \frac{\text{resourceTotal}(p)}{7}\right), \\ \text{selfHarmAvoidance} &= \begin{cases} 0, & \text{if the acting player has a building on } h, \\ 1, & \text{otherwise.} \end{cases} \end{aligned}$$

Hurting the leader is the anti-runaway term, production damage captures future income suppression, theft value captures immediate swing, and self-harm avoidance prevents obvious placements on the acting player's own production.

A.3.11 **accept-or-reject-trade-offers**

$$\begin{aligned} \text{targetAcceptScore} = & 0.50 \cdot \text{selfGain} + 0.25 \cdot \text{leaderPenalty} \\ & + 0.15 \cdot \text{resourceUrgency} + 0.10 \cdot \text{fairnessBalance}. \end{aligned}$$

The current implementation approximates this as a two-option ratio over accept and reject. The accept score is the responder's simplified self-gain:

$$\text{acceptScore} = \text{selfGain} = \text{requestValue} - 0.5 \cdot \text{giveCost},$$

where `requestValue` uses the responder's need map and `giveCost` uses the responder's surplus map. The reject score is

$$\text{rejectScore} = \text{clamp}(1 - \text{acceptScore} + 0.1).$$

A.3.12 **generate-trade-offers**

$$\begin{aligned} \text{tradeOfferScore} = & 0.35 \cdot \text{legality} + 0.45 \cdot \text{selfGain} \\ & + 0.15 \cdot (1 - \text{leaderPenalty}) + 0.05 \cdot (1 - \text{fairnessPenalty}). \end{aligned}$$

A trade must first be legal. Its main purpose is to improve the acting player's position. Helping the leader is strategically risky, and highly unfair offers are less plausible and less likely to be accepted. The components are:

$$\begin{aligned} \text{selfGain} &= \text{clamp}(\text{requestValue} - 0.5 \cdot \text{giveCost}), \\ \text{fairnessPenalty} &= \min(1, \max(0, \text{giveCost} - \text{requestValue})), \\ \text{legality} &= \begin{cases} 0, & \text{if the offer gives resources the player does not own,} \\ 1, & \text{otherwise,} \end{cases} \\ \text{leaderPenalty} &= \begin{cases} 1, & \text{if the target is the visible leader,} \\ 0, & \text{otherwise.} \end{cases} \end{aligned}$$

The generated offer is compared against the benchmark-best score via a ratio.

A.3.13 **select-targeted-trade-partner**

This task is evaluated as an option ratio. Each candidate partner is scored as

$$\text{partnerScore} = \text{selfGain} + \text{availabilityBonus} - 0.4 \cdot \text{leaderHelpPenalty},$$

where

$$\begin{aligned} \text{availabilityBonus} &= \begin{cases} 0.25, & \text{if the partner can provide the requested resources,} \\ 0, & \text{otherwise,} \end{cases} \\ \text{leaderHelpPenalty} &= \begin{cases} 1, & \text{if the partner is the visible leader,} \\ 0, & \text{otherwise.} \end{cases} \end{aligned}$$

This favors partners that produce good value and can satisfy the request, while discounting trades with the leader. Targeted trade proposals emit this task at runtime, and the selected target is compared against all opponent candidates rather than only against itself.

A.3.14 generate-counter-trade-offer

$$\begin{aligned} \text{counterTradeScore} = & 0.35 \cdot \text{legality} + 0.45 \cdot \text{selfGain} \\ & + 0.15 \cdot (1 - \text{leaderPenalty}) + 0.05 \cdot (1 - \text{fairnessPenalty}). \end{aligned}$$

The structural reasoning follows the generated-trade-offer task from Section A.3.12, but now from the responder’s perspective relative to an incoming offer. The payload reuses the trade-quality subscores from Section A.3.12:

$$\begin{aligned} \text{selfGain} &= \text{clamp}(\text{requestValue} - 0.5 \cdot \text{giveCost}), \\ \text{fairnessPenalty} &= \min(1, \max(0, \text{giveCost} - \text{requestValue})), \\ \text{legality} &= \begin{cases} 0, & \text{if the responder gives resources they do not own,} \\ 1, & \text{otherwise,} \end{cases} \\ \text{leaderPenalty} &= \begin{cases} 1, & \text{if the original proposer is the visible leader,} \\ 0, & \text{otherwise.} \end{cases} \end{aligned}$$

The benchmark payload stores this through `offerContext` with `bestOfferScore = 1` rather than logging each candidate counteroffer separately.

A.3.15 city-vs-settlement-vs-road-prioritization

This task is evaluated as an option ratio. It compares build classes rather than board coordinates and asks which legal build family has the highest strategic value at the current state. Let the maxima below range over the currently legal options:

$$\begin{aligned} \text{settlementScore} &= \max \text{settlementScore}(v), \\ \text{roadScore} &= \max \text{roadScore}(e), \end{aligned}$$

where settlement scores use Section A.3.2 and road scores use Section A.3.3.

$$\text{cityScore} = 0.65 \cdot \text{production} + 0.10 \cdot \text{diversity} + 0.05 \cdot \text{portSynergy} + 0.20.$$

Production dominates because a city doubles an existing production site; diversity and port access still matter because they determine how valuable that amplification is, and the constant 0.20 reflects the guaranteed VP and output increase. The task is emitted for settlement, city, and normal road-building choices. Free roads from Road Building are intentionally excluded.

A.3.16 development-card-purchase-decision

This task is evaluated as an option ratio. Buying a development card is an investment choice against competing uses of resources and should be favored when hidden VP, Knight access, or tactical flexibility outweigh immediate board development. This decision is derived from the shared `devCard` plan `baseScore` from Section A.2.5:

$$\text{devCardBaseScore} = \max(0.2, 0.55 \cdot (0.5 + 0.5 \cdot \text{cityPlanScore}))$$

if the development deck is available, otherwise 0. The two options are

$$\text{buy-dev-card} = \text{clamp}(\text{devCardBaseScore}),$$

$$\text{skip-dev-card} = \text{clamp}(\max(\text{saveScore}, B_{\text{nondev}})),$$

where B_{nondev} is the best affordable non-development-card plan baseScore. `buyDevCard` emits this task directly. If development-card purchase was affordable but skipped for the full turn, the negative case is recorded at turn end.

A.3.17 **discard-strategy-after-seven**

This task is scored as

$$\frac{\text{RetainedHandValue}}{\text{BestRetainedValue}}.$$

Discarding is constrained by a fixed number of cards to lose, and the actual strategic objective is to maximize the value of the retained hand rather than the discarded hand. The evaluator enumerates all legal discard bundles summing to $\lfloor \text{total_hand_size}/2 \rfloor$, subtracts each from the current hand, and scores the retained hand with:

$$\text{bestAffordable} = \max \text{ affordable plan baseScore},$$

$$\text{bestFuture} = \max \text{ plan effectiveScore},$$

$$\text{diversity} = \text{clamp} \left(\frac{\text{distinctPositiveResources}}{5} \right).$$

$$\text{retainedHandValue} = \text{clamp}(0.55 \cdot \text{bestAffordable} + 0.30 \cdot \text{bestFuture} + 0.15 \cdot \text{diversity}).$$

The final task score is the selected retained-hand value divided by the best legal retained-hand value. Candidate discard bundles are generated directly from the player hand and required discard count.

A.3.18 **bank-trade-decision**

$$0.50 \cdot \text{resourceValueDelta} + 0.30 \cdot \text{buildUnlock} + 0.20 \cdot \text{opportunityCost}.$$

This task is currently approximated through a choice ratio over candidate bank/port trades plus a no-trade option. For each candidate trade, the evaluator computes the trade ratio using `GameLogic.getTradeRatio` and scores the candidate as

$$\text{tradeScore} = \text{clamp}(\text{need}[\text{get}] - 0.5 \cdot \text{surplus}[\text{give}] + 0.5),$$

and includes a fixed baseline

$$\text{no-trade} = 0.45.$$

Acquiring a needed resource is valuable, paying with a surplus resource is less painful than paying with a scarce one, and the no-trade baseline prevents weak legal trades from looking attractive by default. The named rubric components are not yet separately logged; they are compressed into a single trade desirability heuristic.

A.3.19 decide-pursue-longest-road

$$\begin{aligned} \text{legacyScore} = & 0.40 \cdot \text{roadProgressRatio} + 0.30 \cdot \text{opponentRoadGap} \\ & + 0.20 \cdot \text{resourceAffordability} + 0.10 \cdot \text{VPImpact}. \end{aligned}$$

The stronger formulation is to record a pending task episode, wait 4–6 turns for the evaluated player, and score using snapshot-plus-outcome evidence:

$$\begin{aligned} \text{pursueScore} = & 0.45 \cdot \text{blockingEffectiveness} + 0.30 \cdot \text{opponentProgressThreat} \\ & + 0.15 \cdot \text{selfRoadSynergy} + 0.10 \cdot \text{resourceCost}. \end{aligned}$$

This redesign is preferable because Longest Road decisions are not fully observable from a single action. The live system already uses the delayed v2-style episode. At decision time it snapshots player road length, strongest opponent road rival and their road length, whether either player currently holds Longest Road, player visible score, and roads built so far in benchmark long-term stats.

After a four-turn horizon, decisive resolution, or game end, it computes:

$$\begin{aligned} G_{\text{self}} &= \max(0, L_{\text{self},\text{end}} - L_{\text{self},\text{start}}), \quad G_{\text{target}} = \max(0, L_{\text{target},\text{end}} - L_{\text{target},\text{start}}), \\ R_{\Delta} &= \max(0, \text{roadsBuiltSinceSnapshot}), \\ \text{opponentProgressThreat} &= \text{clamp} \left(\frac{(L_{\text{target},\text{start}} - L_{\text{self},\text{start}}) + 2H_{\text{target},\text{start}} + 2}{8} \right), \\ \text{blockingEffectiveness} &= \text{clamp}(0.45 + 0.18G_{\text{self}} - 0.12G_{\text{target}} + 0.25H_{\text{self},\text{end}}), \\ \text{selfRoadSynergy} &= \text{clamp} \left(\frac{G_{\text{self}}}{4} + 0.35H_{\text{self},\text{end}} \right). \end{aligned}$$

Here each H indicator is 1 when that player has Longest Road at the named time and 0 otherwise. The resource-cost term is

$$\text{resourceCost} = \begin{cases} 1, & \text{successful defer and } R_{\Delta} = 0, \\ 0.5, & \text{pursue and } R_{\Delta} = 0, \\ \text{clamp} \left(\frac{G_{\text{self}} + \text{longestRoadBonus}}{R_{\Delta}} \right), & \text{otherwise.} \end{cases}$$

The final alternatives are

$$\begin{aligned} \text{pursueScore} = & \text{clamp}(0.45 \cdot \text{blockingEffectiveness} + 0.30 \cdot \text{opponentProgressThreat} \\ & + 0.15 \cdot \text{selfRoadSynergy} + 0.10 \cdot \text{resourceCost}), \end{aligned}$$

$$\begin{aligned} \text{deferScore} = & \text{clamp}(0.45 \cdot (1 - \text{opponentProgressThreat}) \\ & + 0.25 \cdot \left(1 - \min \left(1, \frac{G_{\text{target}}}{3} \right) \right) \\ & + 0.15 \cdot \left(1 - \min \left(1, \frac{G_{\text{self}}}{3} \right) \right) + 0.15D_0). \end{aligned}$$

Here D_0 is 1 if $R_{\Delta} = 0$ and 0.2 otherwise. This task now runs as a pending long-horizon episode, and finalization writes one ordinary benchmark task result after the horizon, decisive resolution, or game end.

A.3.20 **decide-pursue-largest-army**

$$\begin{aligned} \text{pursueScore} = & 0.40 \cdot \text{knightCardRatio} + 0.30 \cdot \text{opponentArmyGap} \\ & + 0.20 \cdot \text{devCardAffordability} + 0.10 \cdot \text{VPImpact}. \end{aligned}$$

The rubric stated at the start of Section A.3.20 remains appropriate, but it should be evaluated over a 4–6 turn future window rather than immediately. Largest Army is not about one development-card purchase or one Knight; it is about whether a multi-turn commitment improves the player’s race standing without unacceptable economic damage. A hybrid intent-plus-outcome model is appropriate because pure outcome scoring would overreact to deck order and dice luck, while pure immediate scoring would ignore whether the strategic plan materialized. Like Longest Road, this task is implemented as a pending long-horizon episode. At decision time it snapshots player knights played, strongest opponent army rival and their knights played, whether either player currently has Largest Army, player visible score, player hand size, and development cards bought so far in benchmark long-term stats.

At finalization it computes:

$$C_{\Delta} = \max(0, \text{devCardsBoughtSinceSnapshot}), \quad K_{\Delta} = \max(0, K_{\text{self},\text{end}} - K_{\text{self},\text{start}}),$$

$$\begin{aligned} G_{\text{start}} &= K_{\text{self},\text{start}} - K_{\text{target},\text{start}}, \quad G_{\text{end}} = K_{\text{self},\text{end}} - K_{\text{target},\text{end}}, \\ \text{knightCardRatio} &= \begin{cases} 1, & K_{\Delta} > 0 \text{ and } C_{\Delta} = 0, \\ 0, & K_{\Delta} = 0, \\ \text{clamp}\left(\frac{K_{\Delta}}{C_{\Delta}}\right), & \text{otherwise,} \end{cases} \end{aligned}$$

$$\text{opponentArmyGap} = \text{clamp}\left(\frac{(G_{\text{end}} - G_{\text{start}}) + 3}{6}\right),$$

$$\Delta R = \text{currentHandSize} - \text{snapshotHandSize},$$

$$\text{devCardAffordability} = \text{clamp}\left(0.6 + \frac{\Delta R}{10} - 0.08C_{\Delta}\right),$$

$$\text{vpImpact} = \text{clamp}\left(\frac{\Delta \text{visibleVP} + H_{\text{army},\text{end}}}{3}\right).$$

Here $H_{\text{army},\text{end}}$ is 1 if the player has Largest Army at finalization and 0 otherwise. The final alternatives are

$$\begin{aligned} \text{pursueScore} = & \text{clamp}(0.40 \cdot \text{knightCardRatio} + 0.30 \cdot \text{opponentArmyGap} \\ & + 0.20 \cdot \text{devCardAffordability} + 0.10 \cdot \text{vpImpact}), \end{aligned}$$

$$\begin{aligned} \text{deferScore} = & \text{clamp}(0.35 \cdot (1 - \text{opponentArmyGap}) + 0.25 \cdot (1 - \text{knightCardRatio}) \\ & + 0.20 \cdot R_{\text{safe}} + 0.20 \cdot (1 - H_{\text{army},\text{end}})). \end{aligned}$$

Here R_{safe} is 1 if $\Delta R \geq 0$ and 0.4 otherwise.